

Stoquify Module System Skill Suite Blueprint

Date: 2026-07-14

Repository: E:\ohada saas\Focused projects\stoquify

Governing proposal: docs/new

ideas/STOQUIFY_MODULE_SYSTEM_CURRENT_STATE_AND_FULL_SYSTEM_PROPOSAL_2026-07-14.md

Objective

Provide a validated, installed, dependency-ordered skill system that can implement Stoquify's module control plane without turning navigation, registration intent, provider events, or UI state into authorization truth.

Design Decisions

- Preserve and strengthen the existing `aqstoqflow-module-*` namespace.
- Maintain one canonical orchestrator and one canonical release skill.
- Convert older overlapping skills into compatibility routers instead of silently deleting them.
- Add focused skills for program status, P0 security, enforcement truth, audit/overrides, internal provisioning, provider shadow reconciliation, and repeatable rollout certification.
- Require current code and generated evidence to override stale reports or graph artifacts from another checkout.
- Keep broad enforcement in observe or shadow mode until a named pilot and module certificate pass.

Ordered Suite

Stage	Canonical skill	Outcome
00	<code>aqstoqflow-module-program-status-register</code>	Truthful stage/evidence register
01	<code>aqstoqflow-module-security-prerequisites</code>	P0 tenant, role-grant, and step-up controls
02	<code>aqstoqflow-module-vocabulary-freeze</code>	Canonical module/dependency manifest
03	<code>aqstoqflow-module-enforcement-truth</code>	Versioned effective policy and kill switches
04	<code>aqstoqflow-module-surface-registry-ratchet</code>	Complete surface ownership and ratchet
05	<code>aqstoqflow-module-entitlement-schema</code>	Durable explicit entitlement truth and migration
06	<code>aqstoqflow-module-package-strategy</code>	Immutable dependency-closed packages
07	<code>aqstoqflow-module-subscription-provisioning-engine</code>	Idempotent internal subscription commands
08	<code>aqstoqflow-module-access-guard-contract</code>	Canonical decision before data access
09	<code>aqstoqflow-module-audit-evidence-overrides</code>	Append-only evidence and governed overrides
10	<code>aqstoqflow-module-deactivation-policy</code> plus leakage skill	Safe lifecycle and retained history
11	<code>aqstoqflow-module-workbench-ux-states</code>	Service-owned role-safe Workbench

Stage	Canonical skill	Outcome
12	<code>aqstoqflow-module-provider-shadow-reconciliation</code>	Provider evidence reconciled in shadow mode
13	<code>aqstoqflow-module-enforcement-pilot</code>	Bounded cohort evidence
14	<code>aqstoqflow-module-rollout-certification</code>	Repeatable named-module graduation
15	<code>aqstoqflow-module-release-gates-and-rollback</code>	Current release and rollback certification

Compatibility And Supporting Skills

- `aqstoqflow-module-commercialization-orchestrator` routes to the canonical orchestrator.
- `aqstoqflow-module-release-gates` routes to the canonical release skill.
- `aqstoqflow-module-control-center-packages` routes to package-read-model and Workbench skills.
- `aqstoqflow-module-surface-inventory-gate` remains a report-mode runner owned by the registry stage.
- Module creation lifecycle is a cross-cutting admission checklist.
- Package read model and billing boundary remain supporting contracts.

Per-Skill Semantic Contract

Every skill has valid trigger metadata, governing-proposal precedence, focused scope, repository sources, invariants, ordered workflow, verification, stop conditions, completion evidence, and synchronized `agents/openai.yaml`. Detailed matrices live one level below `SKILL.md`.

Validation

Required before installation:

1. Official `quick_validate.py` for every folder.
2. No TODOs, `$_` placeholders, or malformed code fences.
3. Exact governing-proposal reference in every skill.
4. Valid agent short description and default prompt naming the skill.
5. Every linked reference exists.
6. Fresh-agent forward tests for orchestration, security, enforcement, provider, audit, rollout, and release semantics.

Execution Discipline

- Execute the earliest unblocked stage only.
- Keep one implementation stage active unless write scopes are independent.
- Never refresh a failing baseline merely to obtain a pass.
- Record current evidence, command results, residual risks, and rollback state.
- Do not mark design artifacts as runtime proof.
- Do not commit, push, deploy, reseed, or run destructive migrations without separate authorization.